FOODVERSE AI

Multimodal Food Intelligence Agent

PROJECT PLANNING PHASE

Scope, milestones, resources, risks, and delivery plan

1. Planning Purpose

The Project Planning Phase organizes the work required to build, test, document, and demonstrate Foodverse AI. It identifies the project's scope, work sequence, resources, dependencies, risks, and completion criteria. The plan reflects the development workflow discussed for the project; it is a practical plan rather than a claim that every future task has already been completed.

2. Project Goal

Deliver a demonstrable Foodverse AI prototype that provides food-related assistance through a web interface and backend APIs. The project should be structured, tested, documented, and prepared for online deployment where hosting configuration and required integrations are available.

3. Scope of Work

Included in the project scope

- Set up the Python environment and install the project dependencies.
- Develop and run the FastAPI backend with Uvicorn.
- Create and test the available API endpoints, including recipe generation and implemented food-related features.
- Handle invalid and unusual input with clear responses.
- Connect the frontend to the backend where the corresponding interface and API are implemented.
- Prepare project documentation and demonstrate the application.
- Prepare for deployment and verify the live application where hosting is configured.

Scope boundaries

- Features that are only ideas should not be represented as complete.
- Advanced AI capabilities depend on working model integrations, credentials, and sufficient runtime resources. ● Food freshness or hygiene assistance is not an official safety certification.
- Commercial scale, guaranteed accuracy, and enterprise-grade availability are outside the confirmed prototype scope.

4. Work Breakdown and Milestones

Stage	Main work	Evidence of completion
1. Brainstorming & Ideation	Define the problem, users, and feature ideas.	Ideation document.
2. Requirement Analysis	Record functional, quality, and technical requirements.	Requirements document and dependency list.
3. Project Design	Plan architecture, modules, and request flows.	Design document.
4. Project Planning	Set scope, sequence, resources, risks, and checkpoints.	Planning document.
5. Project Development	Implement backend, feature routes, frontend, and integrations.	Runnable prototype and source code.
6. Project Testing	Test endpoints, input validation, integration, and stability.	Test results and resolved defects.

Stage	Main work	Evidence of completion
7. Project Documentation	Prepare reports, setup steps, and technical notes.	Documentation files.
8. Project Demonstration	Show the application, core flows, and known limitations.	Demonstration and submission materials.

5. Proposed Execution Order

- Confirm the repository structure and current working files.
- Start the backend in the intended Python environment and confirm it launches.
- Test each implemented endpoint with valid and invalid requests.
- Verify frontend-to-backend communication and correct any API URL or CORS issues.
- Run a complete user flow through the interface.
- Prepare deployment settings and secrets outside the source code.
- Deploy backend and frontend as required, then test the public URL.
- Finalize documentation and prepare a short demonstration.

Risk	Mitigation
Dependency installation or Python version conflicts	Use the project virtual environment and verify packages; record errors and resolve them before deployment.
Endpoint returns errors or invalid output	Test valid, empty, whitespace-only, and unusual inputs; keep response formats consistent.
AI service unavailable or missing API key	Store keys in environment variables and provide clear service-unavailable errors.
Frontend cannot reach backend after deployment	Set the deployed API base URL and configure CORS appropriately.
Feature is described as complete when it is not	Document current implementation status and demonstrate only verified behavior.

6. Resources Required

- **Hardware:** a development laptop; the project has been worked on using an HP laptop with an Intel Core i3 7th generation processor and 8 GB RAM.
- **Software:** Python, a virtual environment, terminal, code editor, browser, Git, and GitHub.
- **Backend tools:** FastAPI, Uvicorn, and the packages listed in requirements.txt.
- **Testing tools:** browser-based Swagger UI at /docs and an HTTP client where needed.
- **Online services:** hosting providers and any AI service accounts required by implemented integrations.

7. Roles and Responsibilities

For an individual/student project, the developer is responsible for implementation, dependency setup, endpoint testing, fixing defects, protecting credentials, preparing documentation, and presenting the prototype. If teammates or a supervisor are involved, responsibilities can be divided among backend, frontend, testing, documentation, and review.

8. Risks and Mitigation

Risk	Mitigation
Freshness/hygiene output is misunderstood as a guarantee	Label results as preliminary guidance and avoid definitive food-safety claims.

9. Schedule and Progress Tracking

The work should be tracked by stage rather than by assumed calendar dates. A stage is complete when its evidence of completion exists and the relevant checks pass. Since the project has already progressed through development and several rounds of API testing, the remaining focus is to finish the pending test work, verify end-to-end behavior, prepare the documentation, and deploy if feasible.

- Maintain a short checklist of tasks and mark each only after verification.
- Keep copies of working code and avoid untested broad changes near submission.
- Record known limitations so they can be explained honestly during demonstration.

10. Completion and Acceptance Criteria

- The backend starts reliably using the documented command.
- Implemented API endpoints return expected structured responses for tested inputs.
- Invalid inputs produce controlled errors rather than unhandled failures.
- The frontend can reach the backend for the implemented user flows.
- Any external keys are configured securely and are not committed to GitHub.
- The project documents explain setup, features, tests, limitations, and deployment status.
- If deployment is completed, the public URL is tested from a separate browser session or device.